COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structure and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	08/27/26	**DATE SUBMITTED:**	08/27/26
NAME:	Jeffordniel H. Cabug	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HANDS-ON ACTIVITY NO. 6.1 SEARCHING TECHNIQUES

1. PROCEDURE OUTPUT

Screenshot

main-2.cpp > main()

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "node.h"
#include "searchingAlgo.h"

const int max_size = 50;

int main (){
    // Generate random values
    int dataset[max_size];
    srand(time(0));

    for(int i = 0; i < max_size; i++){
        dataset[i] = rand();
    }
    for(int i = 0; i < max_size; i++){
        std::cout << dataset[i] << " ";
    }
    std::cout << std::endl;

```

Observation

The code generates 50 random integer values, stores them in an array called dataset, and displays all the generated values on the screen. The srand(time(0)) function is used to initialize the random number generator so that different values are generated each time the program runs.

Table 6-1. Data Generated and Observations.

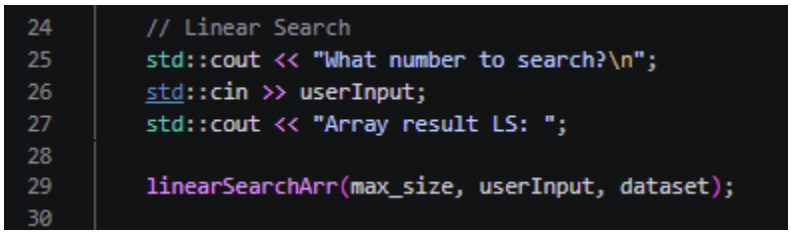
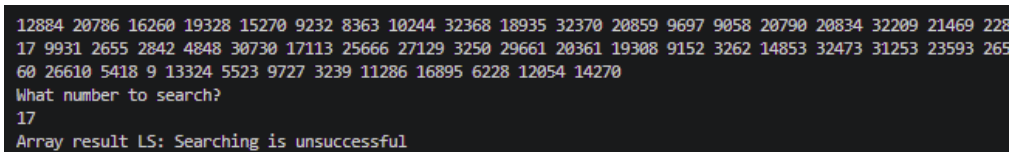
Code	
Output	
Observations	**The code asks the user to enter a number and searches for it sequentially through the array using the linearSearch function, which determines whether the number is present.**

Table 6-2a. Linear Search for Arrays

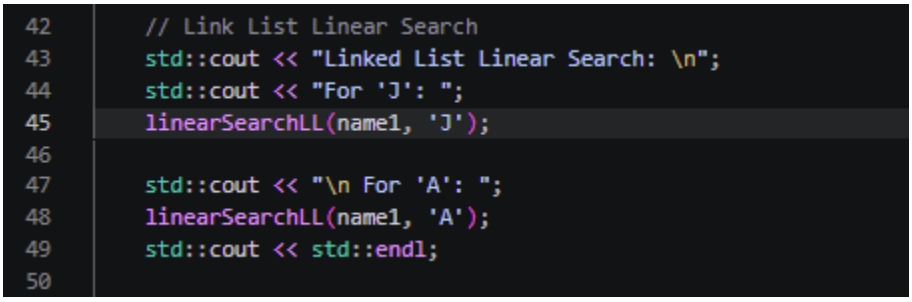
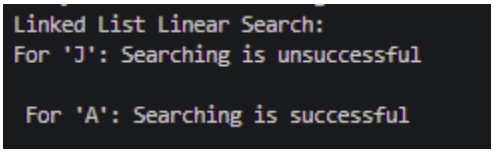
Code	
Output	
Observations	**The code searches the linked list for the characters 'J' and 'A' using the linearSearchLL function and displays whether each character is found.**

Table 6-2b. Linear Search for Linked List

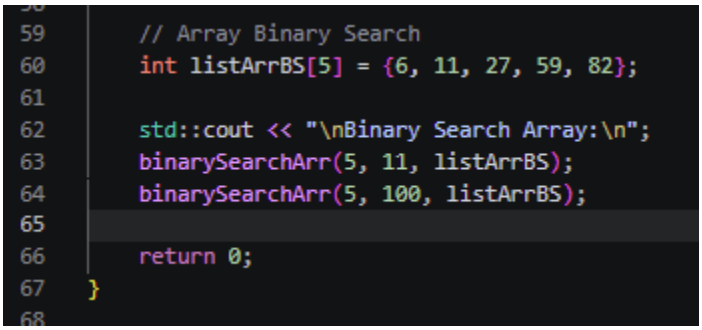
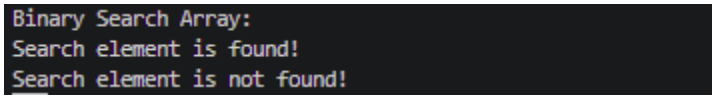
Code	
Output	
Observations	**The binarySearchArr() function is first used to search for the value 11, which is found in the array, resulting in the message "Search element is found!". The function is then used to search for 100, which is not included in the array, so the program displays "Search element is not found!".**

Table 6-3a. Binary Search for Arrays

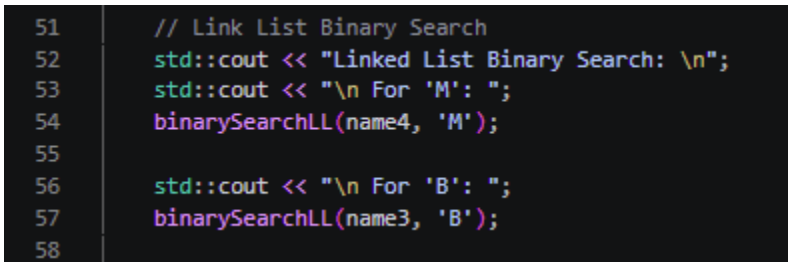
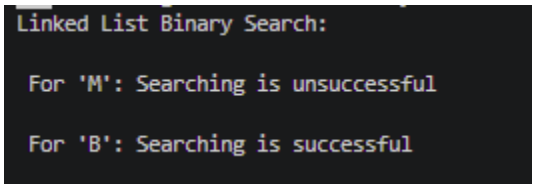
Code	
Output	
Observations	**Using binary search searching for two characters, 'M' and 'B'. The binarySearchLL() function is called first with 'M', and the output shows "Searching is unsuccessful", indicating that the character is not found in the linked list. The function is then called with 'B', and the output shows "Searching is successful", meaning that the character is present in the linked list.**

Table 6-3b. Binary Search for Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1

```cpp
int main (){

    // Problem 1
    int dataset[max_size] = {12, 5, 23, 7, 16, 2, 19, 10, 4, 21};

    // Search for 18
    int comp = 0;
    for(int i = 0; i < max_size; i++){
        comp++;

        if(dataset[i] == 10){
            break;
        }
    }
    std::cout << "Comparisons using Array: " << comp << "\n";

    // Create linked list
    Node<int>* int1 = new_node(12);
    Node<int>* int2 = new_node(5);
    Node<int>* int3 = new_node(23);
    Node<int>* int4 = new_node(7);
    Node<int>* int5 = new_node(16);
    Node<int>* int6 = new_node(2);
    Node<int>* int7 = new_node(19);
    Node<int>* int8 = new_node(10);
    Node<int>* int9 = new_node(4);
    Node<int>* int10 = new_node(21);

    int1->next = int2;
    int2->next = int3;
    int3->next = int4;
    int4->next = int5;
    int5->next = int6;
    int6->next = int7;
    int7->next = int8;
    int8->next = int9;
    int9->next = int10;

    std::cout << "Comparisons using LL: ";
    std::cout << lSLLCount(int1, 10) << "\n\n";

```

Problem 2

```
50      // Problem 2
51      int count = 0;
52
53      for(int i = 0; i < max_size; i++){
54          if(dataset[i] == 21){
55              count++;
56          }
57      }
58      std::cout << "Array occurrences of 21: " << count << std::endl;
59
60      count = 0;
61      Node<int>* current = int1;
62
63      while(current != NULL){
64          if(current->data == 21){
65              count++;
66          }
67          current = current->next;
68      }
69      std::cout << "Linked List occurrences of 21: " << count << "\n\n";
70
```

Problem 3

```cpp
    // Problem 3
    int sorted[max_size] = {2, 4, 6, 9, 11, 13, 15, 17, 20, 24};
    std::cout << "Sorted Array: ";
    for(int i = 0; i < max_size; i++){
        std::cout << sorted[i] << " ";
    }
    std::cout << std::endl;

    // Binary Search for 13
    int low = 0; int up = max_size - 1; int middle; int counter = 1;
    while(low <= up){

        // Compute middle
        middle = (int)((low + up) / 2);

        std::cout << "Repetition " << counter << std::endl;

        if(13 == sorted[middle]){
            std::cout << "Search element is found\n";
            break;
        }

        else if(13 < sorted[middle]){
            up = middle - 1;
        }

        else{

            low = middle + 1;
        }
        counter++;
    }
    std::cout << std::endl;

```

Problem 4

```cpp
    // Problem 4

    if(bSRecursive(0, max_size - 1, 13, sorted)){
        std::cout << "Search element is found\n";
    }
    else{
        std::cout << "Search element is not found\n";
    }

    return 0;
}

```

OUTPUT:

```
 *  Executing task: C:/Windows/System32/cmd.exe /d /c .\build\Debug\outDebug.exe

Comparisons using Array: 8
Comparisons using LL: 8

Array occurrences of 21: 1
Linked List occurrences of 21: 1

Sorted Array: 2 4 6 9 11 13 15 17 20 24
Repetition 1
Repetition 2
Repetition 3
Search element is found

Search element is found
 *  Terminal will be reused by tasks, press any key to close it.
```

CONCLUSION:

The linearSearchArr() function searches for a value in an array by checking each element one by one. If the value is found, it prints that the search is successful. The linearSearchLL() function does the same thing but for a linked list by going through each node. The getMiddle() function finds the middle node of a linked list using slow and fast pointers, which is needed for the linked list binary search. The binarySearchLL() function performs binary search on a sorted linked list by checking the middle node and reducing the search area. The binarySearchArr() function performs binary search on a sorted array by using the low, up, and middle values to find the target faster. The lSLLCount() function is a supplementary function that counts how many comparisons are made while searching through a linked list. Lastly, bSRecursive() is a recursive binary search function that keeps dividing the array into smaller parts until the value is found or there are no more elements to search.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **CHATGPT AND AI OVERVIEW**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of concepts and grammar correction.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Using AI for explanation of the supplementary activity**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

COMPUTER ENGINEERING

QUEZON CITY

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.